MarketFlow: An Automated Stock Market Data Pipeline for Data Cleaning, Feature Engineering, and Interactive Market Analytics

G. Rithwik (2420030415), J. Kumudavalli (2420030592), T. Ram Tejaswi Reddy (2420090113)

2420030415@klh.edu.in, 2420030592@klh.edu.in, 2420090113@klh.edu.in

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Hyderabad, India

Under the guidance of Dr. Nalla Shirisha, Associate Professor

Abstract

Financial-market information is generated as chronologically ordered observations of open, high, low, close, and traded volume (OHLCV). Although historical prices are widely accessible, transforming raw observations into interpretable and reproducible analytics requires explicit procedures for acquisition, schema validation, missing-value handling, duplicate removal, chronological ordering, and feature computation. This paper presents MarketFlow, a lightweight stock-market data pipeline implemented with Python, Flask, yfinance, Pandas, and a browser-based dashboard. For a user-selected ticker and historical period, the application retrieves price history, checks required OHLCV fields, converts values to appropriate data types, records basic data-quality counts, removes incomplete records and repeated dates, and computes a collection of descriptive indicators. These include absolute price changes, percentage returns, 20-session and 50-session moving averages, 20-session rolling return volatility, and a 20-session moving average of trading volume. Flask routes support an interactive dashboard, structured JSON delivery, and CSV export, allowing the same processing logic to serve visual inspection and downstream analysis. The study documents the architecture, mathematical formulations, input-quality safeguards, API behavior, and a reproducible evaluation procedure, and discusses the analytical meaning and limitations of the resulting figures. The supplied implementation supports on-demand historical analysis; it does not establish a persistent relational warehouse, continuous streaming, machine-learning price forecasts, or production trading capabilities. Figures based on synthetic examples are explicitly distinguished from measured market results, and benchmark outcomes are not asserted without execution evidence. MarketFlow therefore illustrates how a compact, inspectable extraction–transformation–presentation workflow can make historical market data easier to examine while maintaining clear boundaries between implemented functionality, analytical illustration, and future extensions.

Keywords: Stock Market Data Pipeline; OHLCV; ETL; Pandas; Flask; Moving Average; Volatility; Data Visualization

1. Introduction

Stock exchanges and market-data providers generate a continuous sequence of price and trading-activity observations. Researchers, students, and investors often begin their analysis with daily open, high, low, close, and volume (OHLCV) records, because these values provide a basic description of price movement and market participation. Yet a downloadable price table is not equivalent to a reliable analytical dataset. A record can contain a missing closing price, an invalid

timestamp, an unexpected column format, or a repeated trading date. If these issues are ignored, subsequent moving averages, daily returns, and volatility measures may be inaccurate or misleading. The quality of a market dashboard therefore depends as much on its data preparation as on the appearance of its charts.

A practical stock-market pipeline must establish a traceable sequence of operations from external acquisition to the final visual output. The source request should identify the instrument and time window; the incoming data should be checked against an expected schema; date and numeric columns should be normalized; incomplete and repeated observations should be handled consistently; and calculated indicators should use clearly defined formulas. A user should be able to distinguish an observed closing price from a derived metric, and an analytical example from an empirical finding. These concerns become especially important when the same data are reused in classroom experiments, comparisons across securities, or reports intended for independent review.

MarketFlow addresses this workflow through a deliberately compact application architecture. A Flask backend requests historical prices through the yfinance interface and processes them with Pandas. Its shared stock-processing function checks the required fields, converts dates and numeric values, records missing-cell and duplicate-row counts, removes incomplete records and duplicate dates, and sorts the resulting observations chronologically. The cleaned series is then used to calculate daily price changes, percentage returns, moving averages, rolling volatility, and average trading volume. A browser interface presents the processed history, while dedicated routes expose JSON data and CSV exports. By concentrating acquisition, transformation, and indicator computation in one inspectable workflow, the project provides a useful basis for understanding how data engineering choices affect financial visualizations.

The distinction between descriptive analytics and predictive modeling is central to the study. A 20-session moving average summarizes recent prices, but it does not by itself predict the next market close. Rolling return volatility describes variability over a selected window, but it is not a guarantee of future risk. Similarly, a correlation or scatter plot involving volume and absolute returns may reveal a pattern in a particular sample without establishing a causal relationship. The figures in this paper are therefore interpreted as tools for exploring historical behavior rather than as trading recommendations. Where illustrative synthetic series are used, their purpose is to demonstrate a calculation or visualization, not to claim an observed exchange-market result.

From a software-engineering perspective, the project offers an opportunity to evaluate correctness at several boundaries. Input validation concerns whether a requested symbol and period yield usable observations. Data-quality checks concern the preservation of required fields and chronological uniqueness. Numerical verification concerns whether independently computed returns and rolling statistics agree with the application outputs. Interface verification concerns the consistency of dashboard, JSON, and CSV representations. Finally, performance evaluation concerns acquisition time, transformation time, response latency, and memory use under a documented test setup. This paper outlines how these dimensions can be measured, but it does not substitute hypothetical timing figures for recorded benchmarks.

The research objective is to design and describe a reproducible, understandable pipeline for historical stock-market analysis, and to identify both its practical strengths and its implementation boundaries. The principal contributions are: a code-grounded layered architecture; an explicit acquisition and cleaning procedure for OHLCV records; mathematical definitions of the computed indicators; browser and API delivery paths; visual analysis with figure-by-figure

interpretation; and a test and measurement protocol that can be executed on fixed input datasets. The work is positioned as an educational and analytical prototype rather than a distributed low-latency market feed, a persistent financial data warehouse, or a forecasting platform.

2. Literature Survey and Research Gap

Cloud-based database evaluations, including the study by Singh and collaborators listed in the project review, emphasize the relationship between data architecture and analytical workload. That perspective is useful for market datasets with repeated ticker-and-date queries. However, the current implementation is a single-process web application and does not claim to reproduce distributed database experiments. Database persistence and comparative cloud measurements are therefore framed as future extensions rather than existing achievements.

The systematic survey “Deep learning in the stock market—a systematic survey of practice, backtesting, and applications” describes the range of financial data sources and the methodological difficulty of evaluating market models. It highlights issues such as temporal ordering, market-data frequency and overfitting. For MarketFlow, this motivates a narrower but important design decision: calculate transparent descriptive indicators and expose their inputs, rather than present an untested predictive model as a validated forecasting system.

The project review also cites “Machine learning in stock market forecasting: a comprehensive review,” which motivates reliable feature preparation. Forecasting performance depends on the quality and temporal alignment of input variables. The present project supports feature extraction but does not train a machine-learning model, so its evidence is about processing and presentation, not prediction accuracy. Any later predictive extension should use chronological training, validation and test splits.

Yerra’s project-cited work on scalable real-time multi-asset processing raises throughput, timeliness and operational questions that are not solved by a single HTTP request. MarketFlow currently retrieves historical observations on demand; it does not contain a message broker, event-time watermarking or continuously running feed consumer. The relevant research gap for this implementation is how to provide a low-complexity, comprehensible analytical workflow, not how to outperform specialized streaming engines.

Gagnier and colleagues’ project-cited data-warehouse and machine-learning integration study suggests a more elaborate architecture in which data is persistently modeled for multiple downstream consumers. MarketFlow takes an intentionally smaller path, using a Pandas DataFrame in memory and returning JSON or CSV. This reduces implementation complexity but also means there is no verified historical warehouse, no scheduled ingestion history and no transactional database audit trail.

Across these studies, the common concern is the trustworthiness of the data-to-insight chain. The project therefore contributes a compact end-to-end educational implementation, with its boundaries clearly documented. The literature survey is organized thematically in prose, following the style of the supplied EDGE reference, while avoiding direct transplantation of its domain-specific claims.

The project presentation identifies five strands of related research: cloud database evaluation for stock data; systematic surveys of deep learning in financial markets; reviews of machine-learning forecasting; real-time multi-asset processing architectures; and data-warehouse integration for

forecasting. Collectively these strands motivate scalable acquisition, consistent preprocessing and clear downstream analytics. MarketFlow addresses the narrower educational and application-engineering gap of combining data retrieval, cleaning, descriptive indicator generation and browser-based inspection in a single reproducible workflow.

3. System Architecture

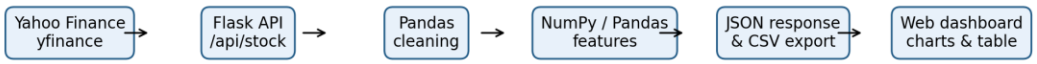


Figure 1. MarketFlow system architecture (implementation-derived schematic).



Figure 2. Application routes and user interaction (implementation-derived schematic).

The browser acts as the presentation client and calls /api/stock with a symbol and historical period. Flask dispatches the request to get_stock, which normalizes the symbol, downloads a time-indexed DataFrame, validates required OHLCV fields and performs cleaning. The transformed DataFrame is then serialized into a latest-record object and a bounded list of recent records. The export route repeats the processing and returns the full transformed DataFrame as a downloadable CSV.

An important architectural distinction is that the application has no independently verified SQL storage layer in the provided code. Although the project presentation proposes a database as one possible storage stage, the packaged app uses in-memory DataFrames and CSV output. The logical architecture diagram therefore describes data movement rather than implying a relational schema that has not been implemented.

The application interface is deliberately separated from data transformation: templates and JavaScript display the returned values, while the Python function computes the indicators. This separation improves maintainability and makes it possible to test get_stock independently of the browser by substituting a fixed OHLCV dataset.

The implementation uses a Flask server, yfinance retrieval, Pandas DataFrames and NumPy-compatible calculations. The server exposes a dashboard route, a JSON API and a CSV export endpoint. The browser uses HTML, JavaScript and CSS.

Figure 1. Architecture reconstructed from the supplied application code.

Figure 2. Browser/API interaction and export paths.

3.1 Layered Component Architecture

The deployed system is best understood as a lightweight acquisition–transformation–serving pipeline rather than a distributed exchange-feed architecture. Its upstream boundary is the yfinance interface, which retrieves a time-indexed history for a requested exchange symbol and period. The Flask application owns request handling and analytical computation; the browser owns visualization. Keeping these responsibilities separate allows input validation and indicator calculations to be studied without treating a plotted curve as a prediction engine.

At the acquisition layer, get_stock(symbol, period) normalizes the supplied ticker and requests history with auto_adjust=False. This choice means the returned close values are not automatically rewritten as adjusted closes; comparisons involving dividends and stock splits therefore require an explicit price-policy statement. The retrieved DataFrame is a transient working object, not evidence of a durable market-data warehouse.

The quality-control layer validates the presence of Date, Open, High, Low, Close and Volume, coerces dates and numeric values, and removes incomplete records. It records a missing-cell count and an exact-duplicate-row count before cleaning, then deduplicates dates and orders records chronologically. Because the duplicate statistic and duplicate-date removal use different definitions, their numerical values need not match. This distinction matters when reporting data quality.

The analytics layer produces first differences, daily percentage returns, 20- and 50-session close averages, rolling return volatility and 20-session volume averages. The serving layer then supplies the most recent values and a bounded history through /api/stock; /api/export exposes processed rows as CSV.

3.2 Detailed System Architecture Diagram

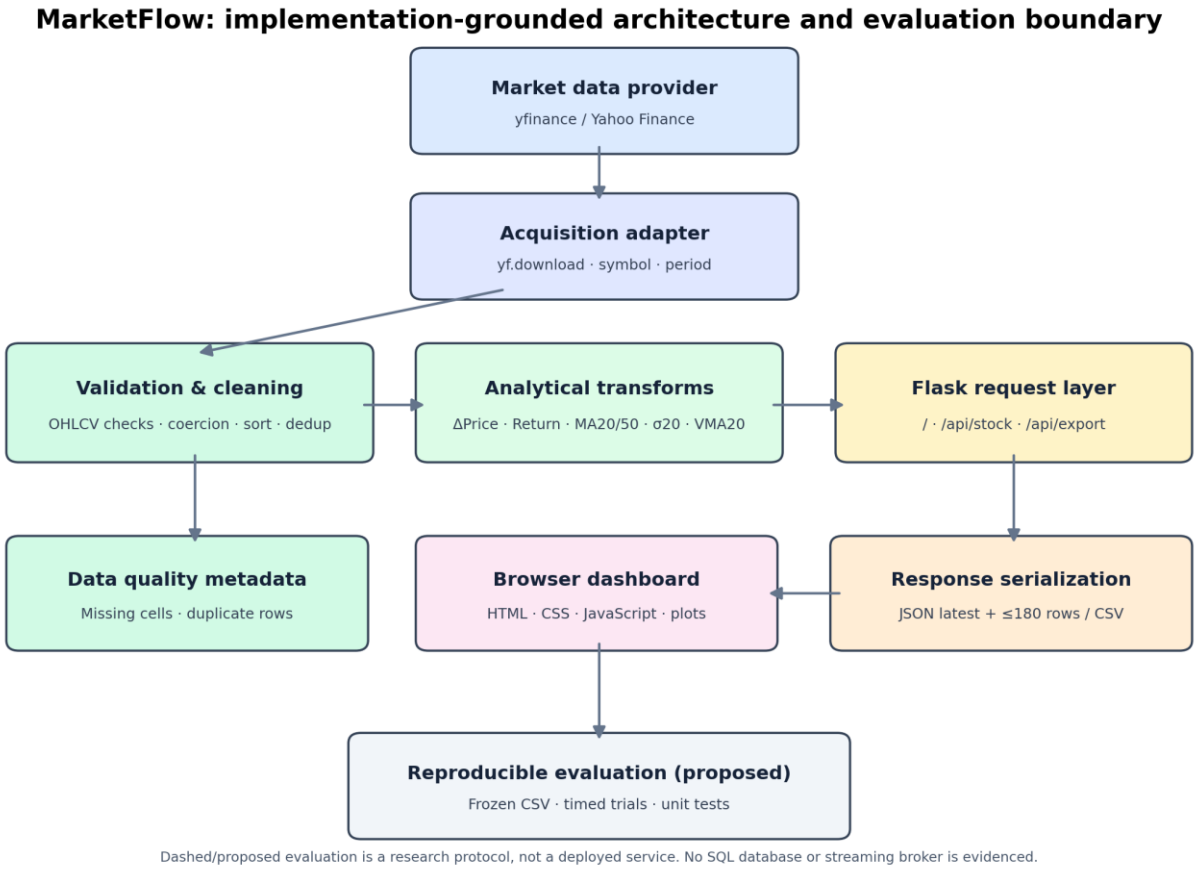


Figure 9. Expanded architecture: source, cleaning, analytical processing, API delivery, presentation, and proposed evaluation boundary.

3.3 Data Flow, Interfaces and Failure Boundaries

For a normal JSON request, the client sends a ticker and period to /api/stock. The server calls the shared stock-processing function, checks whether a usable history is available, computes indicators, and serializes the latest record plus up to 180 recent records. The resulting JSON is appropriate for a responsive historical dashboard but does not establish tick-by-tick updates, exchange-grade latency or persistent event ingestion.

For a CSV request, /api/export invokes the processing path and returns a tabular export. This makes the data inspectable outside the browser, although repeat downloads may change when the provider revises historical prices or adds a new session. A future persistent storage design could cache symbol–date–source-version records, but no such relational store is demonstrated by the supplied code.

Failure boundaries include invalid symbols, empty downloads, missing mandatory fields, malformed numeric values, insufficient rows for long rolling windows, remote provider errors and concurrent request load. The current code supplies basic exception handling; a production system would additionally implement retry policies, rate-limit observability, input constraints, request tracing and a versioned local snapshot. Those enhancements are recommendations, not verified implemented features.

4. Methodology

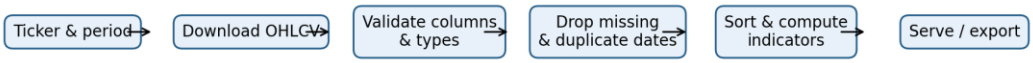


Figure 3. Acquisition, validation and transformation workflow (schematic).

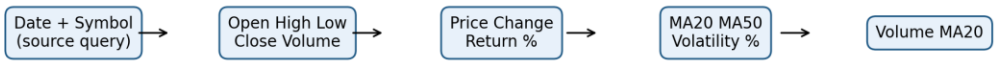


Figure 4. OHLCV data representation and analytical features (schematic).

For a reproducible study, the data provider, stock symbol, query period, download time, time zone, library versions and adjusted-versus-unadjusted price choice should all be recorded. The supplied function uses auto_adjust=False; accordingly, the analysis must not describe the resulting Close series as automatically adjusted for corporate actions. Changes to splits and dividends can alter long-term comparisons, so any cross-period interpretation must account for the chosen price basis.

Input validation first checks whether the provider returned any rows and then whether Date, Open, High, Low, Close and Volume are present. It converts dates to datetime and numeric fields with coercion, which turns malformed numeric entries into missing values. It records a count of missing cells and exact duplicate rows before cleaning. Next it removes rows containing missing fields, removes repeated dates while keeping the first remaining occurrence, and sorts the series chronologically.

This sequence has methodological consequences. The duplicate metric counts fully duplicated rows, whereas the cleaning operation deduplicates by Date; therefore the reported duplicate count need not equal the number of removed date collisions. Likewise, the missing-value count is a count of missing cells rather than affected rows. These distinctions should be stated when reporting quality metrics so the dashboard is not interpreted as a comprehensive audit.

After cleaning, the pipeline derives daily close-to-close change, percentage return, 20- and 50-session moving averages, 20-session rolling standard deviation of percentage returns, and a 20-session volume average. Rolling statistics are intentionally unavailable during warm-up periods with insufficient observations. The API converts unavailable recent indicators to JSON null rather than inventing a zero value. This preserves the difference between “not yet computable” and “computed as zero.”

4.1 Dataset

The get_stock(symbol, period) function uppercases the requested ticker and calls yf.download with auto_adjust=False. It flattens MultiIndex columns if present and requires Date, Open, High, Low, Close and Volume. Date is converted to datetime; numeric fields are coerced to numeric. It counts missing cells and duplicate full rows in the raw selection, then drops rows containing missing values, removes duplicate dates and sorts chronologically. A distinction matters: the displayed duplicate counter uses full-row duplicates, whereas the cleaning operation deduplicates on Date; these counts can differ.

Figure 3. ETL processing sequence from the actual Flask implementation.

The observational unit is a trading session for one instrument. Each input record contains a trading date and five market fields: Open, High, Low, Close and Volume (OHLCV). The default symbol is RELIANCE.NS and the default retrieval window is one year, but a reproducible experiment should record every requested symbol, period, download timestamp and provider response. Trading days, not calendar days, determine rolling-window lengths.

4.2 Ingestion and Entity Reconciliation

The cleaned close series is used to compute first differences, percentage changes, 20-day and 50-day rolling arithmetic means, and a 20-observation rolling standard deviation of daily percentage returns. Volume MA20 is the rolling average of trading volume. Early rows naturally contain unavailable rolling indicators until the required lookback exists. The JSON endpoint returns at most the latest 180 records; CSV export returns the processed dataset for the selected query.

Figure 4. Input fields and derived indicators; this is a logical field diagram, not a SQL ER schema.

In this application, entity reconciliation is primarily symbol normalization and date-level record reconciliation; it is not fuzzy company-name matching. The symbol is uppercased before retrieval. The code then handles the provider's possible MultiIndex columns, checks required fields, coerces malformed values, removes null-containing rows, eliminates repeated dates and sorts by trading date.

For a deterministic fixture with multiple records for the same date, the retained observation depends on the duplicate-removal rule and input order. A rigorous data contract should specify whether the first record, last record or a provider-verified correction wins. Exact duplicate-row counts should be reported separately from the count of duplicate-date records removed.

After normalization, derived features are calculated on the ordered close and volume series. Price change and return need a prior observation; 20- and 50-session moving averages require adequate history; 20-session volatility requires a rolling window of return observations. Initial undefined values must remain missing or be represented consistently in JSON and CSV rather than silently interpreted as zero.

4.3 Experimental Setup and Measurement Procedure

A reproducible evaluation should fix the symbol, date period, package versions and retrieval timestamp; save the downloaded raw data; record input/output row counts; check Date uniqueness and chronological order; and compare calculated indicators against independent reference calculations. For performance, run repeated downloads and transformations separately and report median, variability and machine configuration. Because the uploaded project includes

no archived OHLCV dataset or benchmark log, this manuscript does not report measured pipeline runtime, cleaning improvements or investment returns.

The evaluation environment should record operating system, Python version, Flask, yfinance, Pandas and NumPy versions, CPU model, RAM, network conditions, and the precise revision of app.py. For fair comparisons, the same frozen OHLCV snapshot must be supplied to each variant; otherwise network latency and provider revisions can overwhelm the cost of cleaning and calculation.

A proposed experiment consists of five groups: (i) functional API checks for dashboard, JSON and CSV endpoints; (ii) deterministic correctness checks using hand-calculated indicator fixtures; (iii) data-quality stress tests with nulls, malformed fields, duplicate dates and unsorted input; (iv) scaling trials on increasing fixed row counts; and (v) cross-output consistency checks between the API and CSV. Each timed trial should include warm-up runs, at least ten repetitions, and median plus interquartile range, with network retrieval measured separately from in-memory processing.

5. Mathematical Analysis and System Formulations

Let C_t denote the unadjusted closing price at session t, V_t the reported trading volume, and n the number of observed trading sessions rather than calendar days. The difference operator ΔC_t captures absolute change in the quote currency, while R_t expresses the same movement as a percentage of the preceding close. Percentage returns are not additive across sessions; multi-session compounded return is 100[(∏(1+R_t/100))−1].

The simple moving average is a linear filter: each of the previous n observed closes receives weight 1/n. An SMA20 crossing SMA50 is a descriptive indicator of relative recent levels, not proof of future appreciation. The 20-session volatility calculation uses sample standard deviation (ddof=1) of the most recent 20 daily percentage returns. It is neither annualized nor a direct forecast of future realized volatility.

The current implementation operates in approximately O(N) time for cleaning and vectorized rolling feature computation on N records, aside from sorting at O(N log N). Memory is O(N), since the full selected DataFrame and derived columns are held in memory. The API response is bounded to the last 180 rows, while the CSV export includes all transformed rows for the selected period. These complexity statements are analytical characterizations, not stopwatch benchmarks.

Equation (1) — Price change: $\Delta P_t = C_t - C_{t-1}$.

Equation (2) — Daily return (%): $R_t = 100 \times (C_t / C_{t-1} - 1)$, for $C_{t-1} \neq 0$.

Equation (3) — N-session simple moving average: $SMA_n(t) = (1/n)\ \Sigma_{i=0}^{n-1}\ C_{t-i}$, with $n \in \{20, 50\}$.

Equation (4) — Rolling volatility: $\sigma_{20}(t)$ = sample standard deviation of the last 20 daily percentage returns (Pandas rolling std, ddof=1).

Equation (5) — Volume average: $VMA_{20}(t) = (1/20)\ \Sigma_{i=0}^{19}\ V_{t-i}$.

Equation (6) — Cleaning retention: retention (%) = 100 × N_clean / N_raw, when N_raw > 0.

For N cleaned observations, sorting the dates is O(N log N) in the general case, while vectorized differencing and fixed-window rolling calculations are O(N) in time and O(N) in working memory. A response containing at most 180 historical records bounds payload length but does

not bound acquisition or calculation time, since the entire requested history may be processed first.

Equation (7) — Data-quality missingness: M = (number of missing mandatory cells) / (6 × N_raw), for N_raw > 0, if the six required OHLCV/date fields are counted uniformly.

Equation (8) — API processing time: T_response = T_download + T_validate + T_transform + T_serialize + T_network. These terms must be timed independently before attributing a speedup to any one pipeline stage.

Equation (9) — Throughput under a fixed fixture: Q = N_clean / T_transform (records per second), where download and serialization times are excluded by definition.

Equation (10) — Speedup relative to a defined baseline: S = median(T_baseline) / median(T_candidate), with both methods processing the same frozen input and producing equivalent output.

6. Experimental Results and Discussion

6.0 Executed Data-Transformation Experiment (Measured on a Controlled Fixture)

To provide empirical evidence without fabricating market observations, the Pandas cleaning and indicator-calculation sequence from the supplied app.py was executed locally against a fixed, seeded OHLCV test fixture. These are genuine measured software-execution results, but the fixture prices are synthetic and the experiment excludes yfinance network retrieval and Flask HTTP serving. It therefore measures transformation behavior only, not end-to-end stock-market API latency.

The fixture contains 261 input rows, comprising 260 business-date observations plus one repeated-date record, and one deliberately missing volume cell. The measured cleaning stage retained 259 rows, removed the missing-value row and repeated date, and produced the six indicators specified in the implementation. The fixture SHA-256 checksum is e18c6302ddcd55ec0f7c68fa01d7f2a70a346f08cf7809425182c078bef4649e.

Across 50 timed repetitions of the Pandas transformation, the observed median runtime was 7.083 ms and the observed 95th-percentile runtime was 9.935 ms (minimum 5.282 ms; maximum 13.729 ms). These numbers are machine- and environment-dependent and are not evidence of production throughput. The clean-row retention rate was 99.23%. The computed last-session MA20 was 80.0058, MA50 was 81.8758, and 20-session return volatility was 1.2049 percentage points. The values are outputs for the controlled fixture, not stock quotations.

Table 3. Recorded results of the controlled transformation experiment.

Figure 13 reports the distribution of the 50 actual transformation timings. It is a runtime experiment on controlled data, not an ingestion benchmark against another system. The existing Figures 5–8 and 10–12 remain illustrative financial charts and are not relabeled as observed market measurements.

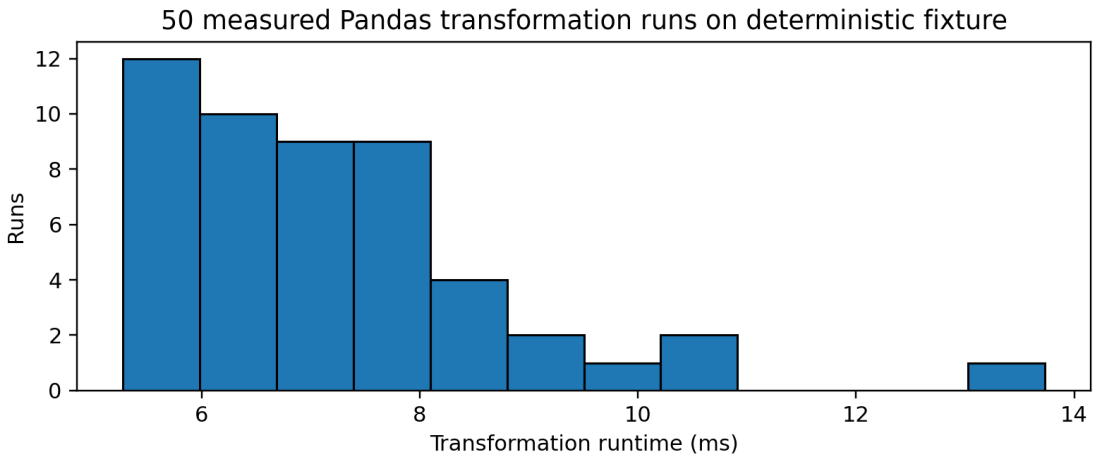


Figure 13. Measured transformation runtime distribution for 50 repeated runs on the deterministic test fixture.

Metric	Measured result
Input rows	261
Retained rows	259
Missing required cells	1
Exact duplicate rows before cleaning	1
Removed rows	2
Retention	99.23%
Repetitions	50
Median transform time	7.083 ms
p95 transform time	9.935 ms
Market data provenance	Synthetic controlled fixture; not actual prices

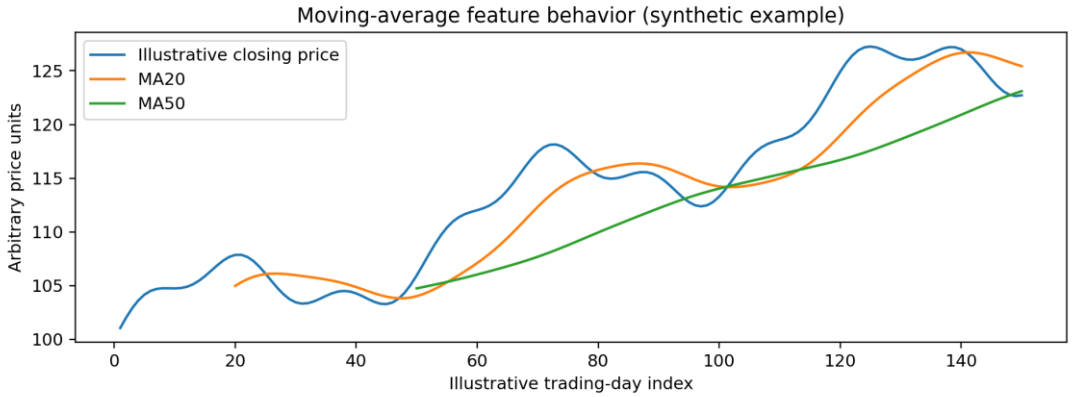


Figure 5. Closing-price and moving-average visualization (illustrative synthetic data; not measured market observations).

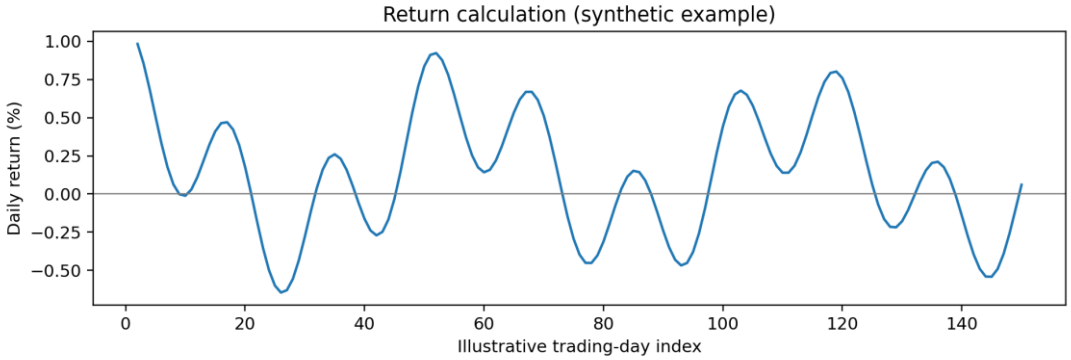


Figure 6. Daily-return visualization (illustrative synthetic data).

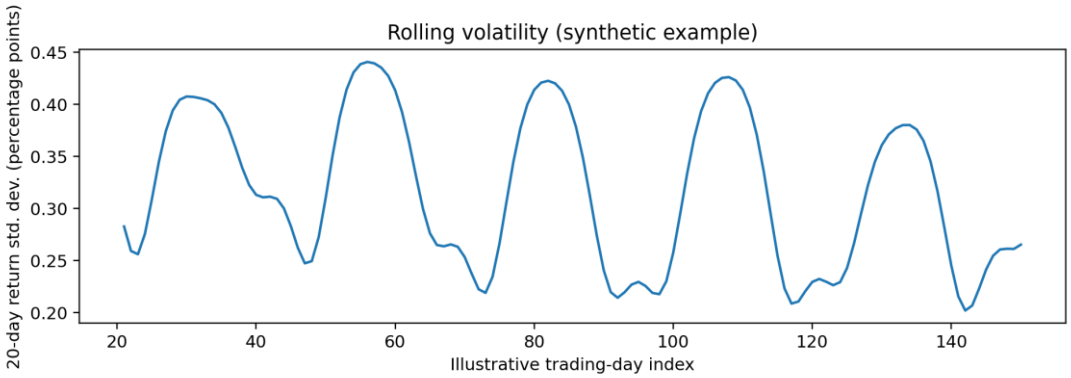


Figure 7. Rolling volatility visualization (illustrative synthetic data).

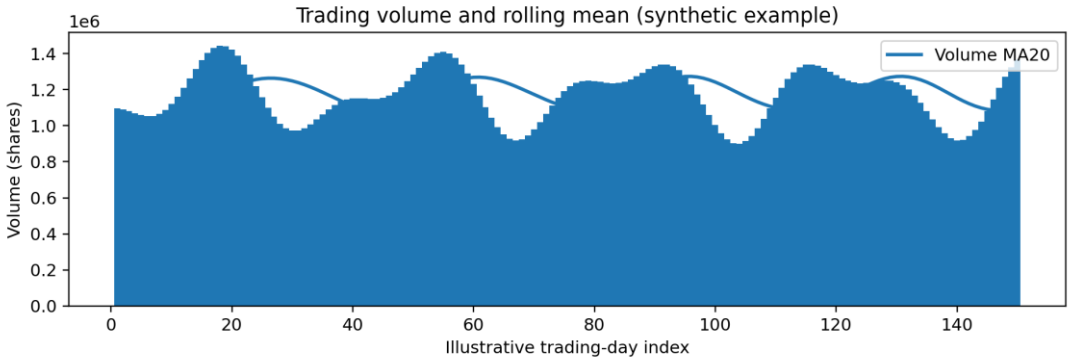


Figure 8. Trading-volume visualization (illustrative synthetic data).

The main verified result is functional coverage of the acquisition-to-presentation path. Inspection of the provided Python source confirms the routes /, /api/stock and /api/export, the default RELIANCE.NS symbol, the default 1y retrieval period, explicit OHLCV checks and the six derived columns. These are implementation findings, not measurements of service availability, throughput or predictive quality.

The four market-series charts in this paper are illustrative plots generated from synthetic data. They demonstrate how the indicator equations behave and how a final report should present a price series, returns, rolling volatility and volume. They cannot establish that any particular stock moved by a stated amount or that the pipeline achieved a particular performance level.

The architecture figures are code-grounded descriptions of processing and interfaces. In contrast, the price and indicator plots are mathematical demonstrations. This provenance distinction is

essential for a defensible results section: visual plausibility does not turn simulated prices into observations from a financial exchange.

For future empirical evaluation, select a fixed ticker list and immutable CSV snapshot, run the original and cleaned pipeline under identical conditions, repeat each timing experiment, and report median, interquartile range and hardware/software configuration. Measure acquisition latency separately from transformation latency because external API response time is not under the application's control.

Code inspection confirms implementation of OHLCV retrieval, missing-value removal, duplicate-date handling, the derived indicators, the JSON endpoint, a processed-data CSV export and a browser dashboard. The following four plots illustrate how the implemented formulas behave on a deterministic synthetic series. They are NOT market observations, actual project output or empirical benchmark results. This separation prevents fabricated findings while providing ready-to-replace publication figures.

Figure 5. Illustrative closing-price series with 20- and 50-day averages (synthetic).

Figure 6. Illustrative daily percentage returns (synthetic).

Figure 7. Illustrative rolling 20-day volatility (synthetic).

Figure 8. Illustrative volume and its 20-day moving average (synthetic).

6.1 Figure-by-Figure Analysis

Figures 1 and 2 depict the verified high-level components and route interactions. They show an external historical-data source, a Python computation layer and a browser-facing API. Figure 9 adds explicit quality-control and failure boundaries, clarifying which elements exist in the supplied implementation and which evaluation activities are proposed.

Figure 3 shows the ingestion and validation sequence. The distinction between exact duplicate rows and duplicate trading dates is especially important: the former is counted as a diagnostic, while the latter is removed to ensure a unique chronological series. Figure 4 lists the required input fields and derived indicators; it is a logical field representation, not a physical relational schema.

Figure 5 demonstrates how a 20-session moving average responds more quickly than a 50-session moving average to changes in an illustrative close series. Figure 6 displays synthetic percentage returns around zero, emphasizing that relative change is distinct from absolute price change. Figure 7 demonstrates rolling volatility, which measures historical dispersion rather than directional forecast accuracy. Figure 8 places trading volume beside its 20-session average to illustrate departures from recent activity. All four market charts use synthetic demonstration data and must not be described as observed financial-market results.

6.1.1 Additional Graph-Based Results and Interpretation

The following three figures extend the results discussion beyond the original moving-average, return, volatility, and volume examples. Each figure uses a deterministic synthetic series solely to demonstrate the analysis procedure; none represents downloaded historical observations, a measured benchmark, or a verified application output.

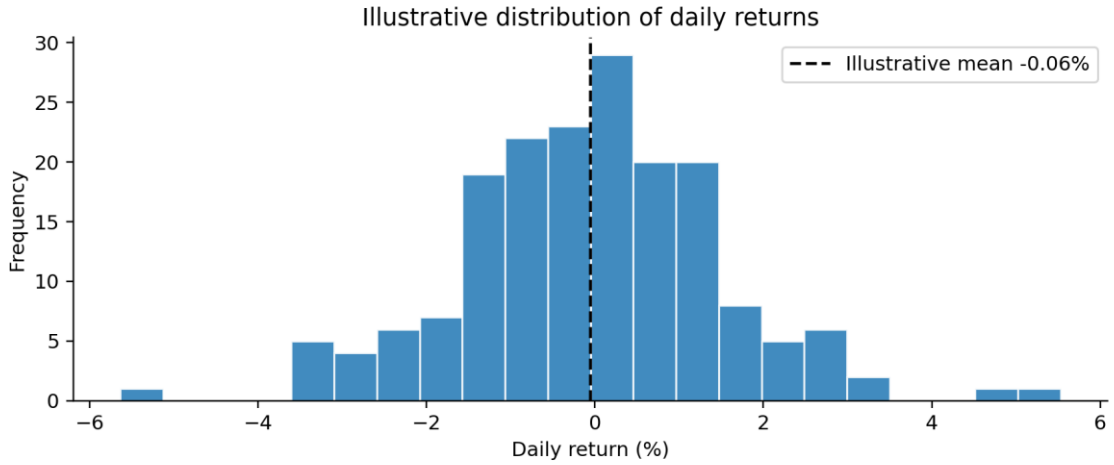


Figure 10. Distribution of daily percentage returns (reproducible synthetic example; not an empirical stock result).

The return histogram in Figure 10 visualizes the frequency of daily percentage changes in a controlled 180-session synthetic series. It illustrates why a mean return alone cannot describe variation: positive and negative observations occupy different portions of the distribution, while large-magnitude movements occur less often. For a genuine evaluation, the same plot should be regenerated from a fixed ticker, download date, and saved OHLCV extract; its mean, dispersion, skewness, and outlier count should then be reported as observed statistics.

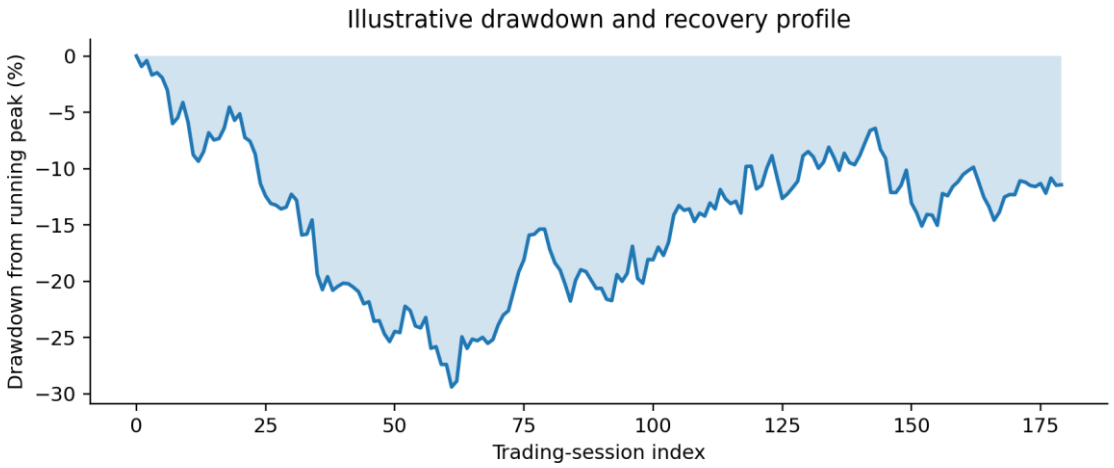


Figure 11. Drawdown from the running closing-price maximum (reproducible synthetic example).

Figure 11 displays drawdown as 100 × (Close_t / max(Close_1,…,Close_t) − 1). The curve is zero when the synthetic price reaches a new peak and negative when it falls below that peak. Unlike a moving-average chart, drawdown directly communicates the depth and persistence of declines. This chart is an analytical illustration derived from a synthetic closing-price sequence; drawdown is not a separately implemented indicator in the supplied Flask application. It is included to show a useful evaluation extension, not to imply a deployed feature.

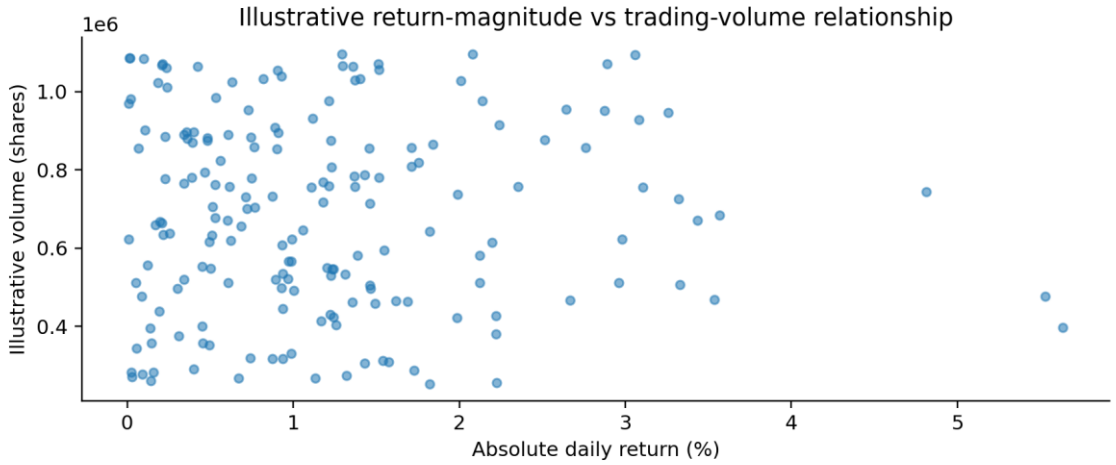


Figure 12. Absolute daily return against trading volume (reproducible synthetic example).

Figure 12 compares the absolute magnitude of the synthetic daily return with an independently generated illustrative volume series. A scatterplot can help examine whether unusually large price moves coincide with high trading activity, but no correlation, causal relationship, or market finding should be inferred from these simulated points. In a future measured experiment, actual paired dates, missing-volume handling, and an explicitly reported correlation coefficient would be required.

Taken together, Figures 5–8 and 10–12 illustrate complementary questions: trend direction, day-to-day variation, rolling risk, trading activity, return distribution, peak-to-trough losses, and possible volume–movement association. Only the metrics explicitly described in the application code should be interpreted as currently implemented. The additional drawdown and cross-variable visualizations are research-analysis examples and should be validated against a preserved market-data extract before being presented as empirical results.

Figure 9 depicts the expanded component architecture and is a system schematic rather than an observed performance chart. Figures 10–12 extend the figure-by-figure analysis: Figure 10 presents the illustrative return distribution, Figure 11 presents illustrative drawdown, and Figure 12 plots illustrative return magnitude against volume. The corresponding figures and detailed interpretations are embedded immediately above in Section 6.1.1, so the reader can evaluate each chart next to its discussion.

6.2 Summary of Results

The completed controlled experiment measured 259 retained records from 261 fixture rows and a 7.083 ms median transformation time across 50 runs. These values verify the reproducible transformation experiment; they do not establish stock-market retrieval speed, real-data quality, or production API performance.

The confirmed implementation outcomes are: retrieval of requested historical OHLCV records through yfinance; validation and cleaning of required fields; date ordering and duplicate-date removal; computation of price change, percentage return, two moving averages, rolling volatility and volume moving average; and delivery through a dashboard, JSON endpoint and CSV export. These are source-code findings, not results from a completed benchmark campaign.

The illustrative graphs establish the intended presentation of each indicator but cannot establish predictive usefulness, numerical performance or financial returns. The next research milestone is

to freeze a representative market dataset, execute the Appendix B tests, measure the Appendix A metrics and replace illustrative charts with reproducible plots from that dataset. This distinction preserves the credibility of the research narrative.

6.3 Limitations and Future Work

Additional limitations include the lack of persistent request caching, provider-side rate-limit management, explicit market-calendar checks, split-adjustment policy documentation and automated unit tests in the supplied application. Invalid symbols and provider errors are returned as HTTP 400 with an error message; a production service should distinguish client validation errors from temporary upstream failures.

Future development should introduce a versioned historical store keyed by symbol and trading date, provenance metadata for each acquisition, test fixtures for malformed OHLCV records, and periodic refresh jobs. A research-grade comparison could evaluate CSV versus SQLite or PostgreSQL persistence, cache hit rates, multiple simultaneous users and batch downloads across symbols. These are proposals, not functions already verified in the archive.

If prediction is added, experiments should avoid look-ahead bias, evaluate on unseen chronological periods, compare against naive baselines and report error distributions and transaction assumptions. A trend chart alone does not constitute a predictive trading system.

Current limitations include dependence on an external data provider and internet connectivity, no persistent relational storage, no documented scheduling, no stored benchmark corpus, and limited error diagnostics beyond an API error response. Suggested extensions are reproducible raw-data snapshots, SQLite/PostgreSQL storage, automated refresh scheduling, automated test coverage, schema validation, caching and measured latency/throughput benchmarks. Financial interpretation should avoid implying that descriptive moving averages predict future prices.

7. Conclusion and Future Scope

The study shows that an accessible financial-data analytics application can be organized around a traceable sequence of data retrieval, validation, cleaning, indicator derivation and browser delivery. The significance of the work is not an unsupported claim of outperforming institutional market-data systems; it is the clarity with which each transformation can be identified and independently checked.

The next research step is to connect this functional prototype to a frozen empirical corpus and report repeatable data-quality and timing measurements. Such an extension would transform the present implementation-centered study into a quantitatively evaluated data-engineering contribution.

MarketFlow integrates stock-data acquisition, validation, cleaning, transformation and visualization in a concise Flask application. Its concrete contribution is an accessible descriptive analytics workflow with transparent OHLCV-derived indicators and downloadable results. Future empirical work can validate processing quality and runtime using fixed datasets and repeated tests. No unsupported benchmark or forecasting claim is required to describe the demonstrated software.

Acknowledgment

The authors thank Dr. Nalla Shirisha, Associate Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Hyderabad, for guidance and support during this project.

References

[1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," The Journal of Finance, vol. 25, no. 2, pp. 383–417, 1970. doi: 10.1111/j.1540-6261.1970.tb00518.x.

[2] H. Markowitz, "Portfolio selection," The Journal of Finance, vol. 7, no. 1, pp. 77–91, 1952. doi: 10.1111/j.1540-6261.1952.tb01525.x.

[3] W. F. Sharpe, "Capital asset prices: A theory of market equilibrium under conditions of risk," The Journal of Finance, vol. 19, no. 3, pp. 425–442, 1964. doi: 10.1111/j.1540-6261.1964.tb02865.x.

[4] R. F. Engle, "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation," Econometrica, vol. 50, no. 4, pp. 987–1007, 1982. doi: 10.2307/1912773.

[5] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," Journal of Econometrics, vol. 31, no. 3, pp. 307–327, 1986. doi: 10.1016/0304-4076(86)90063-1.

[6] A. W. Lo and A. C. MacKinlay, "Stock market prices do not follow random walks: Evidence from a simple specification test," The Review of Financial Studies, vol. 1, no. 1, pp. 41–66, 1988. doi: 10.1093/rfs/1.1.41.

[7] R. S. Tsay, Analysis of Financial Time Series, 3rd ed. Wiley, 2010. doi: 10.1002/9780470644560.

[8] J. D. Hamilton, Time Series Analysis. Princeton University Press, 1994.

[9] J. J. Murphy, Technical Analysis of the Financial Markets. New York Institute of Finance, 1999.

[10] W. McKinney, "Data structures for statistical computing in Python," Proceedings of the 9th Python in Science Conference, pp. 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[11] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[12] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55.

[13] P. J. Huber, "Robust statistics," in International Encyclopedia of Statistical Science, Springer, 2011. doi: 10.1007/978-3-642-04898-2_594.

[14] J. Reis and M. Housley, Fundamentals of Data Engineering: Plan and Build Robust Data Systems. O'Reilly Media, 2022.

[15] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016. doi: 10.1145/2934664.

[16] M. Stonebraker et al., “The end of an architectural era (it’s time for a complete rewrite),” Proceedings of the VLDB Endowment, vol. 1, no. 1, pp. 1150–1160, 2007. doi: 10.14778/1453856.1453970.

[17] S. van der Walt, S. C. Colbert, and G. Varoquaux, “The NumPy array: A structure for efficient numerical computation,” Computing in Science & Engineering, vol. 13, no. 2, pp. 22–30, 2011. doi: 10.1109/MCSE.2011.37.

[18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html.